

Revised Next Push Directive

Sonatina main first: frontend-neutral trace/debug/content-addressing substrate, with Fe as an adapter and Cranelift as a later projection

Prepared May 24, 2026

Executive shift: do not block v1 on Sean's Cranelift or Wasmtime/SP1 work. Overhaul the existing Sonatina #284 branch against sonatina main so that Sonatina itself exposes a frontend-neutral trace/debug/content-addressing substrate. Fe then adapts its OriginExportKey/TraceFact model onto that substrate. Cranelift and other targets plug in later through target-specific projection adapters.

1. What changed since the previous PDF

The prior directive treated Sean's Cranelift branch as a likely base for the next Sonatina instrumentation push. Sean's latest guidance changes that plan: the Cranelift branch has not been pushed recently, depends on a Wasmtime fork for SP1 compatibility work, and is not needed for v1. The first instrumentation PR should target sonatina main and remain backend-change-agnostic.

Decision point	Previous plan	Revised plan
Base branch	Potentially base on Sean's Cranelift branch after compatibility audit.	Target sonatina main. Treat Cranelift/Wasmtime/SP1 as later consumers.
Primary scope	Sonatina trace contract plus possible multi-target branch integration.	Frontend-neutral Sonatina substrate: debug handles, trace views, content addressing, tests.
Cranelift	Maybe included as v1 proof target.	Not required for v1. Preserve future compatibility by API design.
Fe	Possible co-development with Fe CLI examples.	Fe is an adapter/consumer. Keep Sonatina PR independent; add Fe integration after the substrate exists.
#284	Prototype to preserve/adapt.	Overhaul freely. It is our branch and can become the new Sonatina trace/debug/content-addressing PR.

2. Revised thesis

Sonatina main

- > frontend-neutral debug/provenance handles
- > trace-visible SSA/CFG views
- > content-addressed Sonatina shape views
- > backend-agnostic APIs
- > Fe/EVM/Cranelift/other frontends and targets as adapters

The central design rule is that Sonatina should not know Fe-specific identity. Sonatina owns Sonatina IR structure and the preservation/combination of opaque frontend handles. Fe owns the meaning of Fe

origins. Other frontends may own Solidity AST IDs, MLIR locations, DSL syntax node IDs, or other source/proof IR identities.

Layer	Owns	Must not own
Sonatina core	Function/block/instruction/value identity; CFG/SSA structure; local debug handles; propagation rules; content-addressed Sonatina views.	Fe HIR/MIR identity, EVM gas semantics, Cranelift/native artifact meaning, frontend source-map policy.
Frontend adapter, e.g. Fe	Mapping frontend entities into opaque Sonatina origin/debug handles; external origin-key validation.	Sonatina IR identity or target backend emission facts.
Target backend adapter	Final artifact facts: EVM bytecode PCs/gas/size, Cranelift/native/CLIF artifacts, target locations.	Frontend source identity or target-neutral Sonatina structure.
Trace/query/report tools	Derived views and reports over emitted facts.	Compiler truth or identity creation.

3. Scope for the v1 Sonatina main PR

The v1 PR should be small enough for Sonatina review, but broad enough to establish the correct architecture. It should not be a Fe-side instrumentation PR and should not depend on Cranelift.

3.1 Include

- Opaque frontend origin/debug handles: compact IDs and metadata tables, with clear documentation that they are not canonical global source identity.
- Per-function, per-block, and per-instruction trace-view APIs for enumerating Sonatina structure without exposing unstable internals more than necessary.
- CFG visibility: block successors/predecessors or edge enumeration sufficient for downstream loop derivation.
- Instruction-in-block and layout/order views sufficient to relate CFG regions to instruction aggregates.
- Conservative debug-handle propagation rules through basic transforms and inlining where already supported.
- Content-addressed shape views for Sonatina functions, blocks, instructions, and CFG/aggregate structure.
- Tests over Sonatina IR itself, including branch/loop-shaped CFGs, inlining tag propagation, and hash stability/canonicalization cases.

3.2 Exclude from v1

- Cranelift instrumentation or Wasmtime fork integration.
- SP1-specific behavior.
- Fe OriginExportKey types inside Sonatina.
- EVM gas attribution inside Sonatina core.
- Datalog, DWARF, ethdebug, LSP/browser UI.
- Full optimizer provenance for every transformation.
- Compilation caching as the primary API; content addressing should come first, cache usage later.

4. Overhauling #284: preserve concepts, replace shape

PR #284 currently contains useful concepts: per-instruction source locations, inlining debug tags, a compilation cache API, and content-addressed cache keys. Because #284 is ours, we can redo it rather than preserve the exact implementation. Treat it as a parts bin, not the foundation.

#284 concept	Keep?	Revised form
Per-instruction SourceLoc(u32)	Yes, but reframe	DebugLocId / FrontendOriginId: compact opaque handle into a table, not global source identity.
DebugTags for inlining provenance	Yes	Frontend-neutral tags: InlineCallsite, SemanticScope, SyntheticReason, LoweringReason, FrontendLabel, OptimizationNote.
Inliner propagates source locations	Yes	Conservative debug-handle/tag propagation with explicit split/merge/synthetic/unknown semantics over time.
Content-addressed compilation cache API	Later	First expose canonical content-addressed Sonatina views. Build cache keys as consumers later.
Hash function structure + source locations	Revise	Dimensioned hashing: structure, opcode/kind, types, constants/immediates, debug-origin-sensitive, debug-origin-elided, layout-sensitive/elided.
Binary cache serialization	Defer	Only after content-addressing policies are stable.

5. Frontend-neutral debug/provenance model

Sonatina needs to preserve provenance without becoming a frontend-specific provenance database. Use local handles and generic records.

```
pub struct FrontendOriginId(u32);
pub struct DebugLocId(u32);
pub struct DebugTagId(u32);

pub struct FrontendOriginRecord {
  pub external_key: Option<String>,    // frontend-owned encoding
  pub source_span: Option<SourceSpan>, // optional display metadata
  pub display_label: Option<String>,
  pub kind: FrontendOriginKind,
}

pub enum FrontendOriginKind {
  SourceExpr, SourceStmt, SourceDecl, SourceType, SourcePattern,
  Synthetic, SemanticScope, InlineCallsite, Unknown,
}
```

Fe can serialize OriginExportKey into external_key. Another frontend could serialize a Solidity AST ID, MLIR location, or custom DSL syntax node. Sonatina treats the external key as opaque.

```
pub struct DebugLoc {
  pub primary_origin: Option<FrontendOriginId>,

```

```

    pub source_span: Option<SourceSpan>,
    pub confidence: DebugConfidence,
}

pub struct DebugTag {
    pub kind: DebugTagKind,
    pub origin: Option<FrontendOriginId>,
    pub payload: DebugTagPayload,
}

pub enum DebugTagKind {
    InlineCallsite, SemanticScope, SyntheticReason, LoweringReason,
    FrontendLabel, OptimizationNote,
}

```

5.1 Non-negotiable invariant

Sonatina treats frontend origins as opaque.
Each frontend owns the meaning and validation of its own origin keys.
Sonatina owns preservation, combination, propagation, and structural visibility of handles.

6. Trace-view APIs for Sonatina structure

The first PR should expose enough Sonatina structure that downstream tooling can derive loops, compute content addresses, and connect final target artifacts later.

```

pub struct FunctionTraceView<'a> { /* opaque view */ }

pub trait SonatinaTraceView {
    fn functions(&self) -> impl Iterator<Item = FuncRef>;
    fn blocks(&self, func: FuncRef) -> impl Iterator<Item = BlockId>;
    fn instructions(&self, func: FuncRef, block: BlockId) -> impl Iterator<Item = InstId>;
    fn block_successors(&self, func: FuncRef, block: BlockId) -> impl Iterator<Item = CfgEdge>;
    fn inst_debug_loc(&self, func: FuncRef, inst: InstId) -> Option<DebugLocId>;
    fn inst_debug_tags(&self, func: FuncRef, inst: InstId) -> &[DebugTagId];
    fn inst_kind(&self, func: FuncRef, inst: InstId) -> InstKindView;
}

```

Exact Rust names should fit Sonatina style. The important contract is semantic: enumerate functions, blocks, instructions, CFG edges, instruction-to-block membership, and attached debug handles without requiring target backend knowledge.

Trace need	V1 Sonatina support	Why it matters
Function identity	Function handle from Sonatina itself.	Root for content-addressed function views and target projections.
Block identity	Block enumeration per function.	Loop derivation and CFG analysis.
Instruction identity	Instruction enumeration per block.	Instruction aggregates, debug propagation, target emission mapping later.
CFG edges	Successor/predecessor or edge iterator.	Header/body/latch/exit derivation and loop membership.
Debug handles	DebugLocId and DebugTagId attached to instructions.	Frontends can recover source/provenance after transforms.
Instruction kind	Target-neutral instruction kind view.	Shape hashing and coarse reports without target-specific semantics.

7. Debug-handle propagation policy

Propagation must be conservative and explicit. We want useful provenance without pretending to solve full optimizer provenance in v1.

Transform situation	V1 policy	Future refinement
Instruction preserved in same slot	Preserve debug loc/tags.	Record preservation event if trace-event layer is added.
Instruction cloned by inliner	Copy callee tags and prepend/append callsite tag according to chosen call-stack order.	Expose virtual call-stack view.
One-to-one replacement	Copy debug loc/tags if replacement is semantically tied to original; mark as Replacement if tag infrastructure supports it.	Add explicit transform event.
Instruction split into several instructions	Propagate same debug loc/tags to children; optionally add SyntheticReason/LoweringReason to non-primary children.	SplitFrom relation/event.
Several instructions merged	Pick primary debug loc or combine into multiple tags; mark confidence as combined/ambiguous.	MergeFrom relation/event.
Synthetic instruction inserted	Require synthetic/lowering reason when feasible; otherwise UnknownSynthetic.	Typed compiler-event emission.
Instruction deleted/elided	No instruction-level debug loc remains; optional elision record later.	Elision events for optimized-code reports.

8. Content-addressing model for Sonatina

The content-addressing side of #284 should be generalized away from cache-first thinking. Hashes should describe canonical Sonatina structure and be useful for regression diffs, fuzzing, debug comparisons, and future cache/package systems.

Dimensions to support or leave explicitly extensible:

- structure_only
- opcode_kind
- type_sensitive
- constants_sensitive
- debug_origin_sensitive
- debug_origin_elided
- layout_sensitive
- layout_elided
- target_projection_sensitive, later in target adapters

8.1 Aggregation targets

- Instruction local hash: instruction kind, operand shape, type shape, selected dimensions.
- Block aggregate hash: ordered instruction aggregate plus terminator/edge shape.
- Function CFG aggregate hash: canonicalized block/edge/instruction aggregate.

- Loop/subgraph aggregate hash: derived from CFG views downstream or exposed if Sonatina has loop analysis.
- Debug-origin-sensitive hash: changes when frontend origin/debug handles change.
- Debug-origin-elided hash: ignores frontend origins so equivalent lowerings from different frontends can compare structurally.

8.2 Cycles and canonicalization

CFGs and SSA/value-use graphs can contain cycles. V1 should choose a documented policy and test it. The best fit is to treat function/CFG aggregation as graph aggregation with stable canonical traversal plus SCC condensation or iterative fixed-point hashing where needed. Whatever policy is chosen must be deterministic and must not depend on hashmap iteration order or incidental allocation order.

- Canonicalize edge ordering by stable block identifiers within the function view.
- Canonicalize instruction ordering by block order plus instruction order within block for layout-sensitive hashes.
- For layout-elided hashes, use structural canonicalization that is independent of block insertion order where practical.
- For cycles, condense SCCs or use deterministic iterative hashing with bounded convergence and documented fallbacks.
- Include tests where equivalent CFGs built in different insertion orders produce the same structure hash.

9. Multi-frontend story

Supporting other frontends atop Sonatina is a first-class requirement. The Sonatina PR must avoid Fe-only concepts in public APIs.

Frontend	Possible external key	What Sonatina sees
Fe	Serialized Fe OriginExportKey.	Opaque external_key plus generic SourceExpr/Stmt/Decl kind and optional span.
Solidity-like frontend	AST node ID + source unit ID.	Opaque external_key, source span, generic semantic scope tags.
MLIR-based frontend	MLIR location or operation ID.	Opaque external_key; Sonatina does not parse MLIR semantics.
zk/proof DSL	Constraint/proof IR node ID, witness/gate ID.	Opaque external_key and possibly SemanticScope/SyntheticReason tags.

This enables target-neutral Sonatina reports across frontends: same CFG/SSA shape, different frontend origin keys. That is valuable for testing alternate frontends, zk/EVM demos, and future full-stack crypto-oriented workflows.

10. Future backend compatibility without depending on Cranelift now

V1 should not require Cranelift support. It should expose the seams that EVM and future Cranelift adapters will consume.

Sonatina core:
function/block/inst/value/debug/CFG/content-address views

EVM adapter later:
Sonatina inst/block -> bytecode pc/range, opcode, gas, byte size, stack/memory/storage location

Cranelift adapter later:
Sonatina inst/block -> CLIF/native/object artifact, size/category/location when available

Implementation note: if Sean's Cranelift branch later introduces a generic Compile pipeline, the Sonatina trace/content-address APIs should sit below that pipeline and be consumed by each backend adapter. The v1 PR should not take a dependency on local Wasmtime path dependencies, SP1 fixes, or any branch-specific backend implementation.

11. Fe integration after Sonatina PR

After Sonatina main has the frontend-neutral substrate, Fe should consume it through an adapter rather than forcing Fe identity into Sonatina.

Fe HIR/MIR OriginExportKey
-> Fe adapter encodes as frontend external_key
-> Sonatina DebugLocId / DebugTagId
-> Sonatina trace/content-address view
-> Fe TraceFacts
-> EVM target facts / CLI / LSP / Datalog later

Fe-side task	Depends on Sonatina v1?	Purpose
Map Fe OriginExportKey to FrontendOriginRecord.external_key	Yes	Preserve Fe SSOT while using frontend-neutral Sonatina handles.
Emit Sonatina structure TraceFacts from trace views	Yes	Connect MIR/runtime meaning to backend-middle structure.
Emit EVM bytecode projection facts	Partially exists	Map target artifacts to Sonatina inst/block where available.
Loop contents CLI demo	Yes	Show Sonatina CFG-derived loop membership, bytecode/gas/size projection, content hashes.
Gas/source attribution	Later	Query/static/dynamic gas through bytecode PC -> Sonatina -> Fe origin chain.

12. Concrete implementation phases

Phase	Branch/repo	Deliverables	Acceptance
0. Rewrite #284 scope	sonatina main PR branch	Update PR title/description; remove or quarantine cache-first assumptions; document frontend-neutral goals.	Reviewer can understand that Sonatina is not adopting Fe identity.
1. Debug handles	sonatina	DebugLocId/DebugTagId/ FrontendOriginId; metadata tables; attach handles to instructions.	Tests attach handles, retrieve handles, and ensure default/no-location behavior is explicit.

2. Trace views	sonatina	Function/block/inst enumeration; inst-in-block; CFG edges; instruction kind view; debug handles in view.	Tests over straight-line, branch, and loop-shaped functions.
3. Propagation	sonatina	Preserve/copy debug handles through inliner and simple transforms already in scope.	Tests show cloned/inlined instructions preserve virtual callsite metadata.
4. Content addressing	sonatina	Instruction/block/function/CFG hashes with dimensioned policies and deterministic ordering.	Golden tests for name/debug-sensitive vs debug-elided, constants-sensitive, structure-sensitive, and order/cycle determinism.
5. Fe adapter	fe	Encode Fe OriginExportKey as frontend external_key; consume Sonatina trace views into Fe TraceFacts.	Trace validator passes; no raw local IDs or ad hoc strings used as identity.
6. Fe loop CLI	fe	loop-contents command over real trace; content-addressed loop summaries; EVM projection when available.	Sean Fibonacci report derives loop membership from Sonatina CFG facts, not fixture assumptions.

13. Tests to require in v1

- Opaque origin test: Sonatina preserves an external key but never interprets it.
- No-location test: instructions without debug metadata are explicit and do not accidentally inherit previous handles.
- Inline propagation test: cloned instructions carry callee/debug tags plus callsite tag in documented order.
- CFG trace-view test: branch and loop-shaped functions expose blocks, insts, and successors deterministically.
- Content hash dimension test: changing only debug origin changes debug-sensitive hash but not debug-elided hash.
- Constant dimension test: changing immediate/constant changes constants-sensitive hash but not structure-only hash.
- Insertion-order test: equivalent graph built in different internal order has the same canonical structure hash.
- Cycle/SCC test: cyclic CFG hashes deterministically without recursion failure or iteration-order dependence.
- Non-Fe frontend smoke test: attach a fake DSL origin key and prove the same APIs work.

14. Risks and mitigations

Risk	Why it matters	Mitigation
SourceLoc becomes a second source	Would conflict with Fe	Use opaque frontend handles;

of truth	OriginExportKey and other frontend identities.	document that external_key is frontend-owned; Fe adapter validates its keys.
Sonatina API becomes Fe-specific	Would make other frontends harder and weaken Sonatina as shared backend IR.	No Fe types in Sonatina. Add fake non-Fe frontend tests.
Content hashes freeze incidental layout	Bad hashes become hard to change and can mislead fuzzing/regression tooling.	Dimensioned policies, layout-sensitive vs layout-elided modes, deterministic canonicalization tests.
V1 tries to solve full optimizer provenance	Scope explosion and unreliable claims.	Start with conservative propagation: preserved, copied, synthetic, unknown.
Craneflight branch churn blocks progress	Wasmtime/SP1 work is not needed for v1.	Target sonatina main; make backend adapters future consumers.
Cache-first design narrows the system	Content addressing should support more than caching.	Expose content-addressed views first; build compilation cache as later consumer.
Loop introspection still ambiguous	Sean's concrete loop questions need CFG-derived membership.	Trace-view APIs must include CFG edges and inst-in-block; Fe loop CLI should report membership source.

15. Review checklist

1. The PR targets sonatina main and does not depend on Craneflight, Wasmtime fork paths, or SP1-specific code.
2. Sonatina public APIs are frontend-neutral and contain no Fe OriginExportKey or Fe HIR/MIR types.
3. Frontend origin handles are opaque and documented as non-canonical outside the frontend that created them.
4. Trace views expose enough SSA/CFG structure for downstream loop derivation.
5. Content-addressing policies are dimensioned, deterministic, and tested against ordering/cycle pitfalls.
6. Debug propagation is conservative and does not claim full optimizer provenance.
7. The branch provides useful Sonatina-level tests independent of Fe.
8. The follow-up Fe adapter path is obvious: Fe encodes its canonical origin keys into opaque frontend records and emits TraceFacts from Sonatina views.

Appendix A. Updated roadmap

Phase 1: Sonatina main
 frontend-neutral debug handles
 trace-visible SSA/CFG views
 content-addressed Sonatina shape views

Phase 2: Fe adapter
 Fe OriginExportKey -> Sonatina frontend handles
 Sonatina trace views -> Fe TraceFacts

Phase 3: EVM projection
 Sonatina inst/block -> bytecode PC/range
 opcode/gas/byte-size/location facts

Phase 4: CLI/LSP examples

```
loop contents
gas by source
bytecode bloat
content-addressed regression
```

Phase 5: Cranelift projection when stable
Sonatina -> CLIF/native/object facts

Phase 6: Datalog/DWARF/ethdebug
downstream exports and query layers over canonical facts

Appendix B. Sean-facing implications

Sean can continue Cranelift/Wasmtime/SP1 work without being blocked by instrumentation. The Sonatina main PR should give that future backend a stable target-neutral substrate to consume later. The first visible payoff for Sean's Fibonacci question should come from Fe consuming Sonatina trace views and producing a validated loop-contents report: Sonatina blocks and instructions in the loop, EVM bytecode PCs projected from them, gas/byte-size/category summaries, and content-addressed loop hashes.

Target future command shape:

```
fe dev trace loop-contents --input target/fib.trace.jsonl --function main
```

Output should include:

```
membership source: sonatina_cfg
Sonatina blocks/instructions
target projection: evm bytecode PCs, gas, byte size
categories: arithmetic, branch/control, stack/local movement, zext/normalization, unknown
content hashes: cfg.structure, inst.aggregate, category.aggregate, debug-elided
gaps: missing zext causality / missing value storage facts / unmapped target ranges
```

Appendix C. Grounding notes

This revision incorporates Sean's latest guidance: do not require Cranelift for v1; target sonatina main; keep the work backend-change-agnostic. It also reflects that PR #284 is our branch and can be overhauled freely. Public GitHub metadata for #284 shows it is a draft PR from micahscopes:debug-info into fe-lang:main and currently contains per-instruction source locations, DebugTags for inlining provenance, and compilation cache/content-addressing work. Sean's Cranelift branch exists but is no longer a v1 dependency for this push.

References:

- <https://github.com/fe-lang/sonatina/pull/284>
- <https://github.com/sbillig/sonatina/tree/cranelift>